

Toxic Comment Classification and Moderation System

NLP Assignment 1 — Pipeline Execution Report

Roll No: 23110327

GitHub Repository:

https://github.com/tanishwanve-git/NLP_assignment1_23110327

This report walks through the full execution of the moderation pipeline described in the project README — from environment setup and data preparation through model training, threshold calibration, final evaluation, and a live run of the moderation web app. Every step below shows the command that was run, a screenshot of the actual output produced, and a short summary of what that output means.

Step 2: Environment Setup

Before anything else can run, all the Python packages listed in **requirements.txt** need to be installed inside the project's virtual environment. This pulls in the core deep learning stack (PyTorch, Hugging Face Transformers and Datasets), the classical ML tools (scikit-learn, pandas, numpy), the plotting libraries used for EDA (matplotlib, seaborn), and Gradio for the web demo.

Command: `pip install -r requirements.txt`

```
(.venv) (base) tanishwanve@Tanishs-MacBook-Pro nlp_toxic_comment_project % pip install -r requirements.txt
Requirement already satisfied: torch==2.0.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 1)) (2.11.0)
Requirement already satisfied: transformers==4.30.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 2)) (5.16.1)
Requirement already satisfied: datasets==2.12.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 3)) (5.0.1)
Requirement already satisfied: scikit-learn==1.2.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 4)) (1.9.0)
Requirement already satisfied: pandas==2.0.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 5)) (3.0.5)
Requirement already satisfied: numpy==1.24.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 6)) (2.5.2)
Requirement already satisfied: gradio==3.35.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 7)) (6.26.0)
Requirement already satisfied: matplotlib==3.7.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 8)) (3.11.1)
Requirement already satisfied: seaborn==0.12.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 9)) (0.13.2)
Requirement already satisfied: tqdm==4.65.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 10)) (4.70.0)
Requirement already satisfied: joblib==1.2.0 in ./venv/lib/python3.14/site-packages (from -r requirements.txt (line 11)) (1.6.0)
Requirement already satisfied: filelock in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (3.32.5)
Requirement already satisfied: typing-extensions==4.10.0 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (4.16.0)
Requirement already satisfied: setuptools==62 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (81.0.0)
Requirement already satisfied: sympy==1.13.3 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (1.14.0)
Requirement already satisfied: networkx==2.5.1 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (3.6.1)
Requirement already satisfied: Jinja2 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (3.1.6)
Requirement already satisfied: fsspec==0.8.5 in ./venv/lib/python3.14/site-packages (from torch==2.0.0->-r requirements.txt (line 1)) (2026.6.0)
Requirement already satisfied: huggingface-hub<2.0,>=1.5.0 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (1.30.0)
Requirement already satisfied: packaging==20.0 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (26.3)
Requirement already satisfied: pyyaml==5.1 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (6.0.3)
Requirement already satisfied: regex==2025.10.22 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (2026.9.3)
Requirement already satisfied: tokenizers==0.24.0 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (0.23.2)
Requirement already satisfied: typer in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (0.27.2)
Requirement already satisfied: safetensors==0.8.0 in ./venv/lib/python3.14/site-packages (from transformers==4.30.0->-r requirements.txt (line 2)) (0.8.0)
Requirement already satisfied: click<9.0.0,>=8.4.2 in ./venv/lib/python3.14/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (8.1.0)
Requirement already satisfied: hf-xet<2.0.0,>=1.5.2 in ./venv/lib/python3.14/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (1.5.2)
Requirement already satisfied: httpx<1,>=0.23.0 in ./venv/lib/python3.14/site-packages (from huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (0.23.0)
Requirement already satisfied: anyio in ./venv/lib/python3.14/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (4.15.1)
Requirement already satisfied: certifi in ./venv/lib/python3.14/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (2026.7.22)
Requirement already satisfied: httpcore==1.* in ./venv/lib/python3.14/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (1.0.9)
Requirement already satisfied: idna in ./venv/lib/python3.14/site-packages (from httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (3.19)
Requirement already satisfied: h11==0.16 in ./venv/lib/python3.14/site-packages (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers==4.30.0->-r requirements.txt (line 2)) (0.16.0)
Requirement already satisfied: pyarrow==21.0.0 in ./venv/lib/python3.14/site-packages (from datasets==2.12.0->-r requirements.txt (line 3)) (25.0.1)
Requirement already satisfied: dill<0.4.2,>=0.3.0 in ./venv/lib/python3.14/site-packages (from datasets==2.12.0->-r requirements.txt (line 3)) (0.4.1)
Requirement already satisfied: requests==2.32.2 in ./venv/lib/python3.14/site-packages (from datasets==2.12.0->-r requirements.txt (line 3)) (2.34.2)
Requirement already satisfied: xxhash in ./venv/lib/python3.14/site-packages (from datasets==2.12.0->-r requirements.txt (line 3)) (4.0.1)
Requirement already satisfied: multiprocess<0.70.20 in ./venv/lib/python3.14/site-packages (from datasets==2.12.0->-r requirements.txt (line 3)) (0.70.19)
```

Terminal output — dependency installation (part 1)

```

Requirement already satisfied: markupsafe<4.0,>=2.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (3.0.3)
Requirement already satisfied: orjson==3.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (3.12.0)
Requirement already satisfied: pillow<13.0,>=8.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (12.3.0)
Requirement already satisfied: pydantic<3.0,>=2.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (2.13.5)
Requirement already satisfied: pydub<1.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (0.25.1)
Requirement already satisfied: python-multipart<1.0,>=0.0.18 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (0.0.32)
Requirement already satisfied: pyt==2017.2 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (2026.3.post1)
Requirement already satisfied: safethtpx<0.2.0,>=0.1.7 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (0.1.7)
Requirement already satisfied: semantic-version==2.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (2.10.0)
Requirement already satisfied: starlette<2.0,>=1.0.1 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (1.6.0)
Requirement already satisfied: tomkit<0.15.0,>=0.12.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (0.14.0)
Requirement already satisfied: uvicorn==0.14.0 in ./venv/lib/python3.14/site-packages (from gradio==3.35.0->r requirements.txt (line 7)) (0.52.4)
Requirement already satisfied: typing-inspection==0.4.2 in ./venv/lib/python3.14/site-packages (from fastapi<1.0,>=0.115.2->gradio==3.35.0->r requirements.txt (line 7)) (0.4.2)
Requirement already satisfied: annotated-doc==0.0.2 in ./venv/lib/python3.14/site-packages (from fastapi<1.0,>=0.115.2->gradio==3.35.0->r requirements.txt (line 7)) (0.0.5)
Requirement already satisfied: annotated-types==0.6.0 in ./venv/lib/python3.14/site-packages (from pydantic<3.0,>=2.0->gradio==3.35.0->r requirements.txt (line 7)) (0.0.0)
Requirement already satisfied: pydantic-core==2.46.5 in ./venv/lib/python3.14/site-packages (from pydantic<3.0,>=2.0->gradio==3.35.0->r requirements.txt (line 7)) (2.46.5)
Requirement already satisfied: shellingham==1.3.0 in ./venv/lib/python3.14/site-packages (from typer->transformers==4.30.0->r requirements.txt (line 2)) (1.5.4)
Requirement already satisfied: rich==13.8.0 in ./venv/lib/python3.14/site-packages (from typer->transformers==4.30.0->r requirements.txt (line 2)) (15.0.0)
Requirement already satisfied: contourpy==1.0.1 in ./venv/lib/python3.14/site-packages (from matplotlib==3.7.0->r requirements.txt (line 8)) (1.3.3)
Requirement already satisfied: cycler==0.10 in ./venv/lib/python3.14/site-packages (from matplotlib==3.7.0->r requirements.txt (line 8)) (0.12.1)
Requirement already satisfied: fonttools==4.28.2 in ./venv/lib/python3.14/site-packages (from matplotlib==3.7.0->r requirements.txt (line 8)) (4.64.0)
Requirement already satisfied: kiwisolver==1.3.1 in ./venv/lib/python3.14/site-packages (from matplotlib==3.7.0->r requirements.txt (line 8)) (1.5.1)
Requirement already satisfied: pyparsing==3 in ./venv/lib/python3.14/site-packages (from matplotlib==3.7.0->r requirements.txt (line 8)) (3.3.2)
Requirement already satisfied: cloudpickle==3.0 in ./venv/lib/python3.14/site-packages (from joblib==1.2.0->r requirements.txt (line 11)) (3.1.2)
Requirement already satisfied: aiohappyeyeballs==2.5.0 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (2.7.1)
Requirement already satisfied: aiosignal==1.4.0 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (1.4.0)
Requirement already satisfied: attrs==17.3.0 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (26.1.0)
Requirement already satisfied: frozenlist==1.1.1 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (1.8.0)
Requirement already satisfied: multidict<7.0,>=4.5 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (6.7.1)
Requirement already satisfied: propcache==0.2.0 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (0.5.2)
Requirement already satisfied: yarl<2.0,>=1.17.0 in ./venv/lib/python3.14/site-packages (from aiohttp!=4.0.0a0,!4.0.0a1->fsspec[http]<=2026.6.0,>=2023.1.0->datasets==2.12.0->r requirements.txt (line 3)) (1.24.5)
Requirement already satisfied: six==1.5 in ./venv/lib/python3.14/site-packages (from python-dateutil==2.8.2->pandas==2.0.0->r requirements.txt (line 5)) (1.17.0)
Requirement already satisfied: charset-normalizer<4,>=2 in ./venv/lib/python3.14/site-packages (from requests==2.32.2->datasets==2.12.0->r requirements.txt (line 3)) (3.5.1)
Requirement already satisfied: urllib3<3,>=1.26 in ./venv/lib/python3.14/site-packages (from requests==2.32.2->datasets==2.12.0->r requirements.txt (line 3)) (2.7.0)
Requirement already satisfied: markdown-it-py==2.2.0 in ./venv/lib/python3.14/site-packages (from rich==13.8.0->typer->transformers==4.30.0->r requirements.txt (line 2)) (4.2.0)
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in ./venv/lib/python3.14/site-packages (from rich==13.8.0->typer->transformers==4.30.0->r requirements.txt (line 2)) (2.18.0)
Requirement already satisfied: mdurl==0.1 in ./venv/lib/python3.14/site-packages (from markdown-it-py==2.2.0->rich==13.8.0->typer->transformers==4.30.0->r requirements.txt (line 2)) (0.1.2)
Requirement already satisfied: mpmath<1.4,>=1.0 in ./venv/lib/python3.14/site-packages (from sympy==1.13.3->torch==2.0.0->r requirements.txt (line 1)) (1.3.0)
(.venv) (base) tanishwanve@Tanishs-MacBook-Pro nlp_toxic_comment_project %

```

Terminal output — dependency installation (continued)

Result

Every dependency was already satisfied inside the local .venv, confirming a working, compatible environment — including torch 2.11.0, transformers 5.16.1, datasets 5.0.1, scikit-learn 1.9.0, pandas 3.0.5, and gradio 6.26.0. The environment was ready for the rest of the pipeline.

Step 3: Dataset Preparation

`src/data_pipeline.py` downloads the Jigsaw Toxic Comment Classification dataset from the Hugging Face Hub (falling back to a mirrored copy, since the original loading script for `google/jigsaw_toxicity_pred` is no longer supported), removes any duplicate rows, splits the data 80/10/10 into train/dev/test, and runs a leakage audit to make sure the same comment never appears in more than one split.

Command: `python src/data_pipeline.py`

```
(base) tanishwanve@Tanishs-MacBook-Pro nlp_toxic_comment_project % python src/data_pipeline.py
Loading dataset: google/jigsaw_toxicity_pred...
Failed to load from google/jigsaw_toxicity_pred: Dataset scripts are no longer supported, but found jigsaw_toxicity_pred.py
Fallback: Attempting to load 'thesofakillers/jigsaw-toxic-comment-classification-challenge'
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
Total instances loaded: 159571
Deduplicating dataset...
Dropped 0 duplicate rows. New size: 159571
Splitting into Train, Dev, Test...
Train size: 127656
Dev size: 15957
Test size: 15958
Leakage audit passed: No overlapping comments between splits.
```

Class Distribution [Train] (N=127,656)	
Label	Pos Neg Pos% Ratio (neg:pos)
toxic	12248 115408 9.59% 1:9
severe_toxic	1290 126366 1.01% 1:97
obscene	6787 120869 5.32% 1:17
threat	380 127276 0.30% 1:334
insult	6301 121355 4.94% 1:19
identity_hate	1146 126510 0.90% 1:110

Class Distribution [Dev] (N=15,957)	
Label	Pos Neg Pos% Ratio (neg:pos)
toxic	1505 14452 9.43% 1:9
severe_toxic	138 15819 0.86% 1:114
obscene	823 15134 5.16% 1:18
threat	49 15908 0.31% 1:324
insult	772 15185 4.84% 1:19
identity_hate	141 15816 0.88% 1:112

Class Distribution [Test] (N=15,958)	
Label	Pos Neg Pos% Ratio (neg:pos)
toxic	1541 14417 9.66% 1:9
severe_toxic	167 15791 1.05% 1:94
obscene	839 15119 5.26% 1:18
threat	49 15909 0.31% 1:324
insult	804 15154 5.04% 1:18
identity_hate	118 15840 0.74% 1:134

Terminal output — `python src/data_pipeline.py`

Result

159,571 total comments were loaded, with 0 duplicate rows dropped. The split produced 127,656 training, 15,957 dev, and 15,958 test examples, and the leakage audit passed with no overlapping comments between splits. The printed class-distribution tables also make the label imbalance obvious — e.g. *threat* has only ~0.30% positive examples (a 1:334 negative-to-positive ratio), which is why the fine-tuning step later relies on class weighting and focal loss.

Step 4: Exploratory Data Analysis (EDA)

`scripts/run_eda.py` reads the training split and generates a few sanity-check plots: a bar chart of per-label counts, a clean-vs-toxic comment breakdown, and a comment-length histogram.

Command: `python scripts/run_eda.py`

```
(base) tanishwanve@tanishs-MacBook-Pro nlp_toxic_comment_project % python scripts/run_eda.py
--- Exploratory Data Analysis (EDA) ---

Label Counts:
toxic          12248
severe_toxic   1290
obscene        6787
threat         380
insult         6301
identity_hate  1146
dtype: int64
/Users/tanishwanve/nlp_toxic_comment_project/scripts/run_eda.py:24: FutureWarning:
Passing `palette` without assigning `hue` is deprecated and will be removed in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for the same effect.
  sns.barplot(x=label_counts.index, y=label_counts.values, palette="viridis")

Saved 'label_distribution.png' for your report.

Clean Comments: 114676
Toxic Comments: 12980
Saved 'clean_vs_toxic.png' for your report.
Saved 'comment_length.png' for your report.
```

Terminal output — python scripts/run_eda.py

Result

The label counts line up exactly with the `data_pipeline.py` output (toxic: 12,248; severe_toxic: 1,290; obscene: 6,787; threat: 380; insult: 6,301; identity_hate: 1,146). Of the 127,656 training comments, 114,676 are clean and 12,980 are toxic. Three plots — `label_distribution.png`, `clean_vs_toxic.png`, and `comment_length.png` — were saved for the report.

Step 5: Classical Baseline Training

`src/train_baseline.py` trains a TF-IDF + Logistic Regression classifier. This acts as a fast, simple baseline that the fine-tuned transformer model can later be measured against.

Command: `python src/train_baseline.py`

```
(base) tanishwanve@Tanishs-MacBook-Pro nlp_toxic_comment_project % python src/train_baseline.py
Loading data for baseline training...
Extracting TF-IDF features...
Training Logistic Regression (One-Vs-Rest) Baseline...
Evaluating on Dev set...

--- Baseline Results ---
Macro F1-Score: 0.4957
ROC-AUC Score: 0.9708

Classification Report:
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Precision is ill-defined and being set to 0.0 in samples with no
predicted labels. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: Recall is ill-defined and being set to 0.0 in samples with no tru
e labels. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
/opt/anaconda3/lib/python3.12/site-packages/sklearn/metrics/_classification.py:1531: UndefinedMetricWarning: F-score is ill-defined and being set to 0.0 in samples with no tr
ue nor predicted labels. Use 'zero_division' parameter to control this behavior.
  _warn_prf(average, modifier, f'{metric.capitalize()} is', len(result))
      precision    recall  f1-score   support

   toxic          0.58         0.84         0.69       1505
  severe_toxic    0.21         0.87         0.34        138
    obscene      0.61         0.88         0.72        823
    threat       0.18         0.69         0.28         49
    insult       0.50         0.87         0.63        772
 identity_hate    0.19         0.77         0.31        141

   micro avg       0.48         0.85         0.62       3428
   macro avg       0.38         0.82         0.50       3428
  weighted avg       0.53         0.85         0.65       3428
   samples avg     0.06         0.08         0.06       3428

Saving baseline model to disk...
Saved as 'tfidf_vectorizer.pkl' and 'baseline_lr_model.pkl'
```

Terminal output — `python src/train_baseline.py`

Result

On the dev set, the TF-IDF + Logistic Regression (One-vs-Rest) baseline reached a macro F1 of 0.4957 and ROC-AUC of 0.9708. The per-label classification report shows a classic high-recall, low-precision pattern for the rarer labels — e.g. threat has 0.69 recall but only 0.18 precision, and identity_hate has 0.77 recall but only 0.19 precision — meaning the linear model catches most of the toxic examples but also fires a lot of false positives on the minority classes. The trained vectorizer and model were saved to `tfidf_vectorizer.pkl` and `baseline_lr_model.pkl`. (Note: these dev-set numbers are close to, but not identical to, the test-set numbers reported in Step 8's final comparison table — 0.4957/0.9708 here on dev vs. 0.5029/0.9723 on the held-out test set.)

Step 6: LLM Fine-Tuning (DistilBERT)

`src/train_llm.py` fine-tunes a `distilbert-base-uncased` model for multi-label toxicity classification, using Focal Loss ($\gamma = 2.0$) together with per-label positive class weights to counter the heavy imbalance seen in Step 3. Fine-tuning the full model for multiple epochs is too slow on a CPU-only laptop, so this step was run in a hosted GPU notebook instead — the notebook-style `!`-prefixed shell commands and the “+Code / +Markdown” cell buttons visible in the screenshots confirm this. Before training, a small multi-GPU device bug in the script was patched with `sed`.

Commands: `!sed -i 's/model.device/self.args.device/g' src/train_llm.py` then `!python src/train_llm.py`

```
[2]: !pip install -q transformers datasets scikit-learn pandas torch

[3]: # 1. Patch the multi-GPU bug in the script
!sed -i 's/model.device/self.args.device/g' src/train_llm.py

# 2. Run the training script again!
!python src/train_llm.py

Loading data for LLM fine-tuning (distilbert-base-uncased)...
Tokenizing texts...
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
config.json: 100%|████████████████████| 483/483 [00:00<00:00, 1.87MB/s]
tokenizer_config.json: 100%|████████████████████| 48.0/48.0 [00:00<00:00, 276kB/s]
vocab.txt: 232kB [00:00, 8.31MB/s]
tokenizer.json: 466kB [00:00, 28.8MB/s]
Initializing Model...
model.safetensors: 100%|████████████████████| 268M/268M [00:01<00:00, 134MB/s]
Loading weights: 100%|████████████████████| 100/100 [00:00<00:00, 1457.87it/s, Materializing parameters]
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key | Status |
-----+-----+
vocab_projector.bias | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_layer_norm.bias | UNEXPECTED |
vocab_transform.weight | UNEXPECTED |
vocab_transform.bias | UNEXPECTED |
pre_classifier.weight | MISSING |
classifier.weight | MISSING |
pre_classifier.bias | MISSING |
classifier.bias | MISSING |

Notes:
- UNEXPECTED: can be ignored when loading from different task/architecture; not ok if you expect identical arch
```

Notebook output — base checkpoint loaded, model initialized

```

Calculated positive class weights: tensor([ 9.4226, 97.9581, 17.8089, 334.9368, 19.2596, 110.3927])
Starting Training Loop...
{'loss': '14', 'grad_norm': '68.31', 'learning_rate': '4.9e-06', 'epoch': '0.01253'}
{'loss': '2.822', 'grad_norm': '5.628', 'learning_rate': '9.9e-06', 'epoch': '0.02506'}
{'loss': '1.056', 'grad_norm': '7.451', 'learning_rate': '1.49e-05', 'epoch': '0.03759'}
{'loss': '0.993', 'grad_norm': '3.732', 'learning_rate': '1.99e-05', 'epoch': '0.05013'}
{'loss': '0.814', 'grad_norm': '9.927', 'learning_rate': '2.49e-05', 'epoch': '0.06266'}
{'loss': '0.9719', 'grad_norm': '4.739', 'learning_rate': '2.99e-05', 'epoch': '0.07519'}
{'loss': '0.6913', 'grad_norm': '1.848', 'learning_rate': '3.49e-05', 'epoch': '0.08772'}
{'loss': '0.8881', 'grad_norm': '10.86', 'learning_rate': '3.99e-05', 'epoch': '0.1003'}
{'loss': '1.081', 'grad_norm': '1.881', 'learning_rate': '4.49e-05', 'epoch': '0.1128'}
{'loss': '0.6383', 'grad_norm': '1.524', 'learning_rate': '4.99e-05', 'epoch': '0.1253'}
{'loss': '0.768', 'grad_norm': '4.669', 'learning_rate': '5e-05', 'epoch': '0.1378'}
{'loss': '0.6609', 'grad_norm': '6.83', 'learning_rate': '4.999e-05', 'epoch': '0.1504'}
{'loss': '1.074', 'grad_norm': '4.976', 'learning_rate': '4.999e-05', 'epoch': '0.1629'}
{'loss': '0.6602', 'grad_norm': '3.124', 'learning_rate': '4.998e-05', 'epoch': '0.1754'}
{'loss': '0.6396', 'grad_norm': '18.65', 'learning_rate': '4.997e-05', 'epoch': '0.188'}
{'loss': '0.9005', 'grad_norm': '0.792', 'learning_rate': '4.995e-05', 'epoch': '0.2005'}
{'loss': '0.6535', 'grad_norm': '1.521', 'learning_rate': '4.994e-05', 'epoch': '0.213'}
{'loss': '0.6809', 'grad_norm': '8.803', 'learning_rate': '4.992e-05', 'epoch': '0.2256'}
{'loss': '0.661', 'grad_norm': '35.3', 'learning_rate': '4.99e-05', 'epoch': '0.2381'}
{'loss': '0.506', 'grad_norm': '3.966', 'learning_rate': '4.987e-05', 'epoch': '0.2506'}
{'loss': '0.6965', 'grad_norm': '19.92', 'learning_rate': '4.984e-05', 'epoch': '0.2632'}
{'loss': '0.7251', 'grad_norm': '7.546', 'learning_rate': '4.982e-05', 'epoch': '0.2757'}
{'loss': '0.527', 'grad_norm': '5.063', 'learning_rate': '4.978e-05', 'epoch': '0.2882'}
{'loss': '0.504', 'grad_norm': '0.5559', 'learning_rate': '4.975e-05', 'epoch': '0.3008'}
{'loss': '0.6741', 'grad_norm': '3.57', 'learning_rate': '4.971e-05', 'epoch': '0.3133'}
{'loss': '0.4853', 'grad_norm': '5.24', 'learning_rate': '4.967e-05', 'epoch': '0.3258'}
{'loss': '0.4806', 'grad_norm': '5.465', 'learning_rate': '4.963e-05', 'epoch': '0.3383'}
{'loss': '0.5742', 'grad_norm': '1.218', 'learning_rate': '4.958e-05', 'epoch': '0.3509'}
{'loss': '0.6135', 'grad_norm': '8.145', 'learning_rate': '4.954e-05', 'epoch': '0.3634'}
{'loss': '0.7213', 'grad_norm': '5.168', 'learning_rate': '4.949e-05', 'epoch': '0.3759'}
{'loss': '0.4636', 'grad_norm': '0.6186', 'learning_rate': '4.943e-05', 'epoch': '0.3885'}
{'loss': '0.8112', 'grad_norm': '0.3962', 'learning_rate': '4.938e-05', 'epoch': '0.401'}
{'loss': '0.8262', 'grad_norm': '3.041', 'learning_rate': '4.932e-05', 'epoch': '0.4135'}
{'loss': '0.4929', 'grad_norm': '7.399', 'learning_rate': '4.926e-05', 'epoch': '0.4261'}
{'loss': '0.5627', 'grad_norm': '4.509', 'learning_rate': '4.92e-05', 'epoch': '0.4386'}
{'loss': '0.6762', 'grad_norm': '0.9731', 'learning_rate': '4.913e-05', 'epoch': '0.4511'}
{'loss': '0.6103', 'grad_norm': '11.12', 'learning_rate': '4.907e-05', 'epoch': '0.4637'}
{'loss': '0.5457', 'grad_norm': '1.505', 'learning_rate': '4.9e-05', 'epoch': '0.4762'}

```

Notebook output — training loop running (epoch 0 → 0.48)

```
'eval_loss': '0.6138', 'eval_macro_f1': '0.631', 'eval_roc_auc': '0.9872', 'eval_f1_toxic': '0.8081', 'eval_f1_severe_toxic': '0.3721', 'eval_f1_obscene': '0.8313', 'eval_f1_threat': '0.4384', 'eval_f1_insult': '0.7683', 'eval_f1_identity_hate': '0.5679', 'eval_runtime': '34.32', 'eval_samples_per_second': '465', 'eval_steps_per_second': '7.285', 'epoch': '2'})
```

50% ██████████ | 7980/15960 [32:17<26:20, 5.05it/s]

100% ██████████ | 15960/15960 [1:00:34<00:00, 7.30it/s]

Notebook output — epoch 2 evaluation, training continues


```

94%|████████████████████| 235/250 [00:32<00:02, 7.19it/s]
94%|████████████████████| 236/250 [00:32<00:01, 7.18it/s]
95%|████████████████████| 237/250 [00:32<00:01, 7.19it/s]
95%|████████████████████| 238/250 [00:32<00:01, 7.19it/s]
96%|████████████████████| 239/250 [00:33<00:01, 7.21it/s]
96%|████████████████████| 240/250 [00:33<00:01, 7.17it/s]
96%|████████████████████| 241/250 [00:33<00:01, 7.15it/s]
97%|████████████████████| 242/250 [00:33<00:01, 7.13it/s]
97%|████████████████████| 243/250 [00:33<00:00, 7.10it/s]
98%|████████████████████| 244/250 [00:33<00:00, 7.10it/s]
98%|████████████████████| 245/250 [00:33<00:00, 7.14it/s]
98%|████████████████████| 246/250 [00:34<00:00, 7.15it/s]
99%|████████████████████| 247/250 [00:34<00:00, 7.18it/s]
99%|████████████████████| 248/250 [00:34<00:00, 7.20it/s]

{'eval_loss': '0.9269', 'eval_macro_f1': '0.6543', 'eval_roc_auc': '0.982', 'eval_f1_toxic': '0.8121', 'eval_f1_severe_toxic': '0.4242', 'eval_f1_obscene': '0.8358', 'eval_f1_threat': '0.5275', 'eval_f1_insult': '0.7562', 'eval_f1_identity_hate': '0.5703', 'eval_runtime': '34.78', 'eval_samples_per_second': '458.8', 'eval_steps_per_second': '7.188', 'epoch': '4'}
100%|████████████████████| 15960/15960 [1:04:48<00:00, 5.08it/s]
100%|████████████████████| 250/250 [00:34<00:00, 7.23it/s]

Writing model shards: 0%|████████████████████| 0/1 [00:00<?, ?it/s]
Writing model shards: 100%|████████████████████| 1/1 [00:00<00:00, 2.02it/s]
There were missing keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.weight', 'distilbert.embeddings.LayerNorm.bias'].
There were unexpected keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.beta', 'distilbert.embeddings.LayerNorm.gamma'].
{'train_runtime': '3890', 'train_samples_per_second': '131.3', 'train_steps_per_second': '4.103', 'train_loss': '0.4439', 'epoch': '4'}
100%|████████████████████| 15960/15960 [1:04:49<00:00, 4.10it/s]
Evaluating Best Model on Dev Set...
100%|████████████████████| 250/250 [00:34<00:00, 7.24it/s]
{'eval_loss': 0.7297298312187195, 'eval_macro_f1': 0.6588378587921561, 'eval_roc_auc': 0.9863774059141076, 'eval_f1_toxic': 0.8105228105228105, 'eval_f1_severe_toxic': 0.44029850746268656, 'eval_f1_obscene': 0.8324056895485467, 'eval_f1_threat': 0.5121951219512195, 'eval_f1_insult': 0.7632275132275133, 'eval_f1_identity_hate': 0.5943775100401606, 'eval_runtime': 34.6521, 'eval_samples_per_second': 460.492, 'eval_steps_per_second': 7.215, 'epoch': 4.0}
Saving Fine-tuned LLM...
Writing model shards: 100%|████████████████████| 1/1 [00:00<00:00, 2.01it/s]
Saved to './fine_tuned_llm'

```

Notebook output — epoch 4 evaluation, training complete, model saved

Result

The pretrained distilbert-base-uncased checkpoint loaded with the expected MISSING/UNEXPECTED key warnings (normal, since the classification head isn't part of the pretrained weights). Training ran for 4 epochs (train_runtime $\approx$ 3,890s), with the training loss falling from $\approx$ 14 at the very first step down to a final train_loss of 0.4439. On the dev set, the best checkpoint reached a macro F1 of 0.6588 and ROC-AUC of 0.9864, with per-label F1 scores of: toxic 0.81, severe_toxic 0.44, obscene 0.83, threat 0.51, insult 0.76, and identity_hate 0.59. The fine-tuned model was saved to ./fine_tuned_llm.

Step 7: Threshold Calibration

`src/calibrate_thresholds.py` loads the fine-tuned model, runs it on the dev set, and sweeps a Precision-Recall curve per label to find the decision threshold that maximizes F1 for each of the six labels individually, instead of using one flat 0.5 cutoff for everything.

Command: `python src/calibrate_thresholds.py`

```
(base) tanishwanve@Tanishs-MacBook-Pro nlp_toxic_comment_project % python src/calibrate_thresholds.py
Loading Dev Data...
Loading Model...
Loading weights: 100% | 104/104 [00:00<00:00, 5168.70it/s]
Running inference on cpu...

Optimizing Thresholds...
toxic      : Optimal Threshold = 0.50 (F1 = 0.8105)
severe_toxic : Optimal Threshold = 0.30 (F1 = 0.5096)
obscene    : Optimal Threshold = 0.45 (F1 = 0.8353)
threat     : Optimal Threshold = 0.15 (F1 = 0.5536)
insult     : Optimal Threshold = 0.55 (F1 = 0.7645)
identity_hate : Optimal Threshold = 0.45 (F1 = 0.5946)

Optimal thresholds saved to thresholds.json
```

Terminal output — `python src/calibrate_thresholds.py`

Result

The optimal per-label thresholds found were: toxic 0.50, severe_toxic 0.30, obscene 0.45, threat 0.15, insult 0.55, and identity_hate 0.45. The rarer labels (threat, severe_toxic, identity_hate) ended up with noticeably lower thresholds than 0.5, which makes sense given how few positive examples they have in training. These values were saved to `thresholds.json` for use during evaluation and inference.

Step 8: Final Model Evaluation

src/evaluate.py is the final, fair comparison: it scores both the classical TF-IDF + Logistic Regression baseline and the fine-tuned DistilBERT model on the held-out test set (data neither model has seen before), at both the default 0.5 threshold and the calibrated per-label thresholds from Step 7.

Command: python src/evaluate.py

Terminal output — `python src/evaluate.py`

Result

DistilBERT clearly beats the classical baseline: macro F1 rises from 0.5029 (baseline) to 0.6813 (DistilBERT, default threshold), and ROC-AUC improves from 0.9723 to 0.9870. Moving to calibrated per-label thresholds (0.6781 macro F1) costs a negligible amount of overall macro F1 in exchange for large gains on the rarest classes — identity_hate F1 rises from 0.2536 (baseline) to 0.5455 (calibrated DistilBERT), and threat rises from 0.3154 to 0.5505.

Result

DistilBERT clearly beats the classical baseline: macro F1 rises from 0.5029 (baseline) to 0.6813 (DistilBERT, default threshold), and ROC-AUC improves from 0.9723 to 0.9870. Moving to calibrated per-label thresholds (0.6781 macro F1) costs a negligible amount of overall macro F1 in exchange for large gains on the rarest classes — identity_hate F1 rises from 0.2536 (baseline) to 0.5455 (calibrated DistilBERT), and threat rises from 0.3154 to 0.5505.

Step 9: Live System Demo (Web App)

app.py ties all three pipeline stages together into an interactive Gradio app: Stage 1 (DistilBERT) scores each of the six toxicity labels, Stage 2 checks whether the comment is a hostile personal attack versus something more benign like venting or critique, and Stage 3 either blocks the comment outright (for direct personal attacks) or rewrites it into a cleaned, non-toxic paraphrase (for everything else).

Command: `python app.py` → opened at `http://127.0.0.1:7860`

Example 1: “This food was so fucking good!”

The screenshot shows a web application titled "Toxic Comment Moderation System". It features an input field for a comment, a "Clear" button, and a "Submit" button. Below the input field, there's a section for "Toxicity Probabilities" showing scores for six categories: toxic (85%), obscene (83%), insult (17%), severe_toxic (10%), threat (2%), and identity_hate (2%). The "Detected Intent" section shows "Emotional_Venting (Fine-Tuned DistilBERT)". The "Decision" section shows "Detoxified (Maintained Intent)". The "Explanation" section shows "Detected Intent: Emotional_Venting. Cleaned profanity while preserving feedback." The "Cleaned Paraphrase" section shows "This food was so extremely good!". At the bottom, there's a "Flag" button and a list of example comments.

Stage 1: DistilBERT scores toxicity across 6 categories. Stage 2: Intent Detection checks for constructive vs hostile intent. Stage 3: Cleans profanity if the intent is not a personal attack.

Input Comment

This food was so fucking good!

Clear Submit

Toxicity Probabilities

toxic 85%

obscene 83%

insult 17%

severe_toxic 10%

threat 2%

identity_hate 2%

Detected Intent

Emotional_Venting (Fine-Tuned DistilBERT)

Decision

Detoxified (Maintained Intent)

Explanation

Detected Intent: Emotional_Venting. Cleaned profanity while preserving feedback.

Cleaned Paraphrase

This food was so extremely good!

Flag

Examples

This login button is completely broken and slow! You are a total idiot and a complete loser, go away! This food was so good! I am so annoyed by this weather today. This project is helpful!

Web UI — venting comment, detoxified rather than blocked

Result

Scored toxic 85% and obscene 83%, but Stage 2 classified the intent as `Emotional_Venting` rather than an attack, so Stage 3 detoxified it instead of blocking it, producing: “This food was so extremely good!” — the positive sentiment is preserved, only the profanity is removed.

Example 2: “You are a total idiot and a complete loser, go away!”

Toxic Comment Moderation System

Stage 1: DistilBERT scores toxicity across 6 categories. Stage 2: Intent Detection checks for constructive vs hostile intent. Stage 3: Cleans profanity if the intent is not a personal attack.

Input Comment

You are a total idiot and a complete loser, go away!

ClearSubmit

Toxicity Probabilities

toxic

83%

insult

78%

obscene

51%

severe_toxic

5%

identity_hate

4%

threat

2%

Detected Intent

Hostile_Attack (Heuristic Rules)

Decision

Blocked

Explanation

Detected intent: Hostile_Attack. Contains harassment or hate speech.

Cleaned Paraphrase

Blocked due to personal attack.

Flag

Examples

This login button is completely broken and slow!You are a total idiot and a complete loser, go away!This food was so good!I am so annoyed by this weather today.This project is helpful!

Web UI — direct personal attack, blocked

Result

Scored toxic 83%, insult 78%, obscene 51%. Stage 2's heuristic rules flagged this as a Hostile_Attack, so Stage 3 blocked the comment outright instead of trying to clean it.

Example 3: “shut up you gay”

NLP Assignment 1 (Roll No: 23110327) — Pipeline Execution Report

Page 13

Toxic Comment Moderation System

Stage 1: DistilBERT scores toxicity across 6 categories. Stage 2: Intent Detection checks for constructive vs hostile intent. Stage 3: Cleans profanity if the intent is not a personal attack.

Input Comment

shut up you gay

Clear

Submit

Toxicity Probabilities

identity_hate

identity_hate

toxic

insult

obscene

severe_toxic

threat

87%

85%

62%

45%

13%

3%

Detected Intent

Hostile_Attack (Heuristic Rules)

Decision

Blocked

Explanation

Detected Intent: Hostile_Attack. Contains harassment or hate speech.

Cleaned Paraphrase

Blocked due to personal attack.

Flag

Examples

This login button is completely broken and slow!

You are a total idiot and a complete loser, go away!

This food was so good!

I am so annoyed by this weather today.

This project is helpful!

Web UI — identity-based hate speech, blocked

Result

identity_hate was the top-scoring label at 87% (toxic 85%, insult 62%). Again classified as a Hostile_Attack and blocked, showing the moderation layer also catches identity-based hate speech, not just generic insults.

NLP Assignment 1 (Roll No: 23110327) — Pipeline Execution Report

Page 14

Example 4: “go away you retard”

Toxic Comment Moderation System

Stage 1: DistilBERT scores toxicity across 6 categories. Stage 2: Intent Detection checks for constructive vs hostile intent. Stage 3: Cleans profanity if the intent is not a personal attack.

Input Comment

go away you retard

Clear

Submit

Toxicity Probabilities

toxic

71%

insult

62%

obscene

14%

identity_hate

5%

severe_toxic

3%

threat

2%

Detected Intent

Hostile_Attack (Fine-Tuned DistilBERT)

Decision

Blocked

Explanation

Detected intent: Hostile_Attack. Contains harassment or hate speech.

Cleaned Paraphrase

Blocked due to personal attack.

Flag

Examples

This login button is completely broken and slow!

You are a total idiot and a complete loser, go away!

This food was so good!

I am so annoyed by this weather today.

This project is helpful!

Web UI — ableist slur, blocked by the fine-tuned intent classifier

Result

Scored toxic 71%, insult 62%. This time it was Stage 2's fine-tuned DistilBERT intent classifier (rather than the heuristic rules) that flagged Hostile_Attack, and the comment was blocked.

Overall takeaway

Across all four examples the moderation layer behaves as intended: comments that vent frustration or express strong sentiment without targeting a person are detoxified and handed back to the user, while direct personal attacks — whether plain insults or identity-based slurs — are blocked outright, regardless of whether the heuristic rules or the fine-tuned classifier is the one that catches them.

NLP Assignment 1 (Roll No: 23110327) — Pipeline Execution Report

Page 15